

## PRACTICAL-04

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
GNU nano 7.2 wait_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;

    for(i = 0; i < 3; i++) {
        pid = fork();

        if(pid == 0) {
            printf("Child %d created with PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d completed\n", i + 1);
            exit(0);
        }
    }

    printf("Parent waiting using wait()...\n");

    for(i = 0; i < 3; i++) {
        wait(NULL);
    }

    printf("All child processes completed using wait().\n");

    return 0;
}
```

```

}
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];

    for(int i = 0; i < 3; i++) {
        pid[i] = fork();

        if(pid[i] == 0) {
            printf("Child %d created with PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d completed\n", i + 1);
            exit(0);
        }
    }

    printf("Parent waiting using waitpid()...\n");

    for(int i = 0; i < 3; i++) {
        waitpid(pid[i], NULL, 0);
    }

    printf("All child processes completed using waitpid()...\n");

    return 0;
}

```

Result:

```

root@SRIVALLI:~# nano wait.c
root@SRIVALLI:~# gcc wait.c -o wait
root@SRIVALLI:~# ./wait
Child 1 created with PID = 648
Parent waiting using wait()...
Child 3 created with PID = 650
Child 2 created with PID = 649
Child 1 completed
Child 3 completed
Child 2 completed
All child processes completed.
root@SRIVALLI:~# |

```

```

root@SRIVALLI:~# gcc waitpid.c -o waitpid
root@SRIVALLI:~# ./waitpid
Child 1 created with PID = 667
Child 2 created with PID = 668
Parent waiting using waitpid()...
Child 3 created with PID = 669
Child 1 completed
Child 2 completed
Child 3 completed
All child processes completed.

```

Report:

Successfully created multiple child processes and synchronized their execution using wait() and waitpid(). Observed that wait() waits for any child process, whereas waitpid() allows waiting for a specific child process.

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
GNU nano 7.2 no_zombie.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if(pid == 0) {
        printf("Child Process PID: %d\n", getpid());
        exit(0);
    }
    else {
        wait(NULL); // Parent waits for child
        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if(pid == 0) {
        printf("Child Process PID: %d\n", getpid());
        exit(0);
    }
    else {
        wait(NULL); // Parent waits for child
        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```

Result:

```
root@SRIVALLI:~# nano zombie.c
root@SRIVALLI:~# gcc zombie.c -o zombie
root@SRIVALLI:~# ./zombie
Parent PID: 762
Child Process PID: 763
Child exiting...
root@SRIVALLI:~# nano no_zombie.c
root@SRIVALLI:~# gcc no_zombie.c -o no_zombie
root@SRIVALLI:~# ./no_zombie
Child Process PID: 782
Zombie process eliminated.
root@SRIVALLI:~# ps -el | grep Z
F S  UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY
root@SRIVALLI:~# |
```

#### Report:

Successfully created and observed a zombie process and eliminated it using the wait() system call. Zombie processes occur when a child process terminates but its parent does not collect its exit status. Proper synchronization using wait() prevents zombie processes and ensures efficient process management.